DESARROLLO DE APLICACIONES II

Respuesta a Tránsito — integración con RabbitMQ

Equipo 9 — Core · Desarrollo de Aplicaciones II

27 de octubre de 2026

Contenido

Respuesta a Tránsito — integración con RabbitMQ . . . 3
1. Cómo nos conectamos — ⚠️ a definir en la reunión . . . 3
2. Cómo está organizada la mensajería . . . 3
3. Cómo saben cuál es su cola . . . 4
4. Quién crea las colas . . . 5
5. Qué pasa si Tránsito está caído . . . 5
6. Ambientes separados — ⚠️ a definir en la reunión . . . 6
Resumen para la reunión . . . 6
Mientras tanto, para no quedarse parados . . . 7

> **Aviso de actualización — 2026-10-17:** para el servidor desplegado, usar integracion-aws.md. Este documento conserva información anterior y no debe usarse como guía operativa definitiva. Las afirmaciones sobre identidad AMQP, ambientes, reintentos y respuestas deben interpretarse según la guía actual.

Respuesta a Tránsito — integración con RabbitMQ

> Las preguntas 2, 3, 4 y 5 ya están resueltas y se pueden implementar hoy. Las preguntas **1 y 6 (conexión y ambientes) todavía no tienen respuesta**: son una decisión que hay que tomar en la reunión. Más abajo va nuestra propuesta para que salga rápido.

---

1. Cómo nos conectamos — ⚠️ a definir en la reunión

Hoy no hay un RabbitMQ desplegado. El Core corre en una máquina de desarrollo con `guest:guest@localhost:5672`. No hay nada a lo que Tránsito se pueda conectar todavía.

Lo decimos derecho porque es lo que los bloquea, y no queremos que armen el publisher contra una URL que no existe.

Nuestra propuesta, para que la reunión sea corta:

	Propuesta	Por qué
Dónde	RabbitMQ en Railway (plugin)	El equipo ya usa Railway; nos da control total sobre usuarios y vhosts, que los servicios gratuitos administrados no dan
Credenciales	**Un usuario por módulo**	Si se filtra el de un equipo, se rota solo el suyo. Con uno compartido hay que avisarle a los 9.
Cómo se las pasamos	Variable de entorno `RABBITMQ_URL`, con todo adentro	Igual que la base de datos. Una sola variable.

La URL que recibirían sería así:

```
RABBITMQ_URL=amqps://transito:<password>@<host>:5671/<vhost>
```

Con eso ya tienen host, puerto, usuario, contraseña y vhost en una sola variable. No necesitan nada más.

Lo que necesitamos de ustedes en la reunión: confirmar que Railway les sirve, o proponer otra cosa.

---

2. Cómo está organizada la mensajería

Hay **un solo exchange para publicar**, compartido por los 9 módulos:

```
muni.inbox        tipo fanout       ← acá publican TODOS
```

Ustedes publican ahí y listo. **No eligen destino** — no les hace falta saber quién consume sus eventos.

Internamente el Core usa otros, pero **no los tocan**:

Exchange	Para qué
`muni.events`	Por donde el Core reparte a cada módulo
`muni.retry.*`	Los escalones de reintento
`muni.dlx`	Lo que agotó los reintentos

Cómo publicar

```
exchange:      muni.inbox
tipo:          fanout
routing key:   ""  (se ignora, es fanout)
persistente:   sí  (delivery_mode = 2)
content-type:  application/json
```

El cuerpo es el sobre común. Está documentado en eventos.md, pero el resumen es:

```
{
  "eventId": "<UUID nuevo por evento>",
  "eventType": "infractionConfirmed",
  "eventVersion": "1.0",
  "occurredAt": "2026-10-01T12:34:56-03:00",
  "sourceModule": "transito",
  "correlationId": "<el del evento que estás respondiendo, si aplica>",
  "data": { }
}
```

Tres cosas que rechazan el evento si se las saltean:

- `occurredAt` **necesita el offset de zona horaria**. Sin él da 422.
- `eventId` es un UUID **nuevo por evento**. Es la clave de idempotencia.
- `sourceModule` tiene que ser `transito`. Publicar en nombre de otro da 403.

---

3. Cómo saben cuál es su cola

Su cola es `q.transito` y les llega **solo lo que pidieron**, no todo el tráfico de la plataforma. El filtrado lo hace el Core.

Hoy están suscriptos a **10 tipos** (según lo que declararon en el board):

```
debtSettled              paymentRegistered        paymentReversed
streetClosureRequested   ticketUpdated            treePruningScheduled
treeRiskDetected         urbanServiceScheduled    workOrderCompleted
workOrderScheduled
```

> **Ojo con esto para armar los listeners:** es **una sola cola con 10 tipos de evento adentro**, no una cola por tipo. O sea que sí tienen que mirar el campo `eventType` de cada mensaje para despacharlo al handler que corresponda — pero nunca les va a llegar un tipo al que no se suscribieron.

Si quieren sumar o sacar tipos, lo hacen ustedes desde el dashboard del Core o por API, sin pedirnos nada:

```
POST   /api/v1/subscriptions      { "eventType": "..." }
DELETE /api/v1/subscriptions/{id}
GET    /api/v1/subscriptions      ← ver las suyas
```

---

4. Quién crea las colas

Las crea el Core, ustedes no declaran nada.

El Core deriva la topología de la tabla de suscripciones y la aplica sobre RabbitMQ al arrancar, y de nuevo cada vez que alguien agrega una suscripción. Su cola ya va a existir cuando se conecten.

En el consumidor, declárenla en **modo pasivo** («ya existe, no la crees»):

```python
# Python / aio-pika
queue = await channel.declare_queue("q.transito", durable=True, passive=True)
```

```javascript
// Node / amqplib
await ch.checkQueue("q.transito");   // checkQueue, NO assertQueue
```

Por qué importa: si la declaran ustedes con argumentos distintos a los del Core (por ejemplo sin el `x-dead-letter-exchange`), RabbitMQ rechaza la declaración con `PRECONDITION_FAILED` y les cierra el canal. Con `passive` / `checkQueue` eso no puede pasar.

No hay que coordinar nada antes de que arranquen a programar. Su cola existe desde que están en el registry, que ya es el caso.

---

5. Qué pasa si Tránsito está caído

Los mensajes esperan. No se pierden.

La cola es `durable` y los mensajes se publican como `persistent`, así que sobreviven incluso a un reinicio del broker. Cuando Tránsito vuelve a conectarse, empieza a consumir lo acumulado.

Pero sí tienen que preocuparse por los duplicados

La entrega es **at-least-once**, como en cualquier broker. Un mensaje puede llegarles más de una vez si:

- hacen `nack` y el Core lo reintenta;
- se cae la conexión después de procesar pero antes del `ack`.

Guarden los `eventId` que ya aplicaron y salgan temprano si se repite. Es la misma regla que aplicamos nosotros: el Core no rutea dos veces el mismo `eventId`, pero eso no cubre un reintento de entrega hacia ustedes.

Y el orden no está garantizado

Dentro de la cola el orden se respeta, pero **un mensaje que falla y se reintenta vuelve más tarde**, después de los que venían atrás. Si tienen lógica que depende del orden (por ejemplo «no procesar `paymentReversed` antes que `paymentRegistered`»), no se apoyen en el orden de llegada: usen los datos del evento.

Cómo manejar los errores

Situación	Qué hacer
Procesado bien	`ack`
Error transitorio (su DB cayó, timeout)	`nack(requeue=false)`
Error permanente (el payload no les sirve)	`nack(requeue=false)`

Nunca `requeue=true`**.** La cadena de reintentos la maneja el Core: **5 s → 30 s → 2 min**, y después el mensaje queda en la DLQ donde un operador lo ve y puede reintentarlo a mano.

> **Corregido el 2026-10-27.** Antes decía `15s → 1m → 5m → 15m`: era un `.env` local de demo, no lo desplegado. La cadena configurada tiene un cuarto escalón de 10 min, pero con el `maxAttempts` por default (4, que cuenta **entregas**) no se llega a usar. Los escalones reales están en `GET /api/v1/topology`, con el `x-message-ttl` de cada cola de espera; si no coinciden con esto, gana el endpoint. Si ustedes reencolan, el mensaje gira en su cola sin backoff y sin quedar auditado.

Una aclaración importante

Todo esto vale **porque su cola ya existe**. Un mensaje publicado hacia una cola que no existe sí se descarta. Como `q.transito` se crea desde sus suscripciones y ya están registrados, no es un problema — pero si algún día dan de baja todas sus suscripciones, los eventos de esos tipos dejan de guardarse para ustedes.

---

6. Ambientes separados — ⚠️ a definir en la reunión

Tampoco está resuelto, y tienen razón en preocuparse: con un solo RabbitMQ, un evento de prueba de Tránsito le llega al Rentas de producción.

Nuestra propuesta: un solo servidor RabbitMQ con **dos vhosts**.

```
amqps://transito:<pass>@<host>:5671/test     ← su rama develop
amqps://transito:<pass>@<host>:5671/prod     ← su rama main
```

Un vhost es un namespace completamente aislado: exchanges, colas y mensajes separados. Un evento publicado en `/test` **no puede** llegar a `/prod`, ni por error de configuración.

Es más barato que levantar dos servidores y la separación es igual de estricta. Lo único que cambia para ustedes es el final de la `RABBITMQ_URL`, que ya les viene distinta por ambiente desde las variables de Railway.

Resumen para la reunión

#	Pregunta	Estado
2	Exchange	✅ `muni.inbox`, fanout, uno para todos
3	Su cola	✅ `q.transito`, con sus 10 tipos filtrados por el Core
4	Quién la crea	✅ El Core. Ustedes usan `passive` / `checkQueue`
5	Si están caídos	✅ Esperan en la cola. At-least-once: necesitan idempotencia
1	Conexión	⚠️ **Falta desplegar RabbitMQ.** Propuesta: Railway, un usuario por módulo
6	Ambientes	⚠️ **Falta decidir.** Propuesta: dos vhosts (`/test` y `/prod`)

Con las 2 decisiones tomadas, les pasamos la `RABBITMQ_URL` de cada ambiente y pueden reemplazar el `LoggingEventPublisher` ese mismo día: lo demás ya está.

Mientras tanto, para no quedarse parados

Los listeners se pueden terminar y testear sin esperar el deploy, porque lo único que falta es la URL. Dos opciones:

Levantar RabbitMQ local (es lo que usamos nosotros):

```
brew install rabbitmq && brew services start rabbitmq
# RABBITMQ_URL=amqp://guest:guest@localhost:5672/
# consola: http://localhost:15672 (guest/guest)
```

O publicar por HTTP, que pasa por el mismo pipeline que AMQP:

```
POST /api/v1/events
Authorization: Bearer <token>
```

El token sale de `POST /api/v1/auth/module-token` con el secret de máquina de Tránsito, que les podemos pasar hoy mismo.

Cualquier duda, el contrato completo del sobre está en eventos.md.